SUZ E07 SLua advanced - objects

[10:41] Cass Vaher: hello everyone :)

[10:42] Suzanna: hello Cass :)

[10:43] xaka Chayoo: Hello all

[10:43] Suzanna: hello Xaka :)

[10:43] Suzanna: hi Ali :)

[10:44] Avril Vallejo: hello all

[10:44] Suzanna: hello Avril :)

[10:44] Ali: HI Suzanna. Hi Xaka, Anais, Taiven, Cass, and Avril.

[10:44] Avril Vallejo: sorry suzanna

[10:44] Avril Vallejo: Hello sungali

[10:45] Anaïs: Hey Ali, Suzanna, all

[10:45] Suzanna: hello Anaïs :)

[10:46] Suzanna: hello Liam :)

[10:46] Liam Hoffen: Hi everyone!

[10:46] Ali: Hi Liam.

[10:46] Cass Vaher: hey ali, liam

[10:47] Ali: BRB. swapping viewers.

[10:49] Ali laughs. "Darn autosit on touch that can't be turned off with the LL viewers."

[10:52] Suzanna: hello Tyra :)

[10:52] Taiven Janick: Hi everyone !

[10:52] Tyra: Hi Suzanna, Ali, and everyone!

[10:53] Tyra: relogging real quick

[10:53] Suzanna: hello Stoney :)

[10:53] Stoney Crystal: hi

[10:53] Ali: Hi TIray\

[10:54] Avril Vallejo: Hello

[10:54] Ali: Hi Stoney.

[10:54] Tyra: (brb!)

[10:54] Ali: Hello Joel

[10:54] joel dawn: Hello Ali, all

[10:54] Suzanna: hello Joel :)

[10:55] joel dawn: today i got it tru)

[10:55] joel dawn: hello Suzanna)

[10:55] Suzanna: hello Marie :)

[10:55] Ali: HI Marie.

[10:55] Marie: Hi Suzanna, Ali, everyone :)

[10:58] xaka Chayoo: Back, hello to all I missed

[10:58] Suzanna: hello Xia :)

[10:58] Suzanna: today we will write a flatvector, or vector 2D, using our (more or less) beloved metatables :)

[10:59] Xia: Hiya Suzanna :D

[10:59] Ali: Hi Xia.

[10:59] Xia: Hiya Ali :)

[11:00] Suzanna: hello Anton :)

[11:00] Anton: Hello Suzanna, everyone

[11:00] Suzanna: starting in a minute :)

[11:01] Suzanna: Welcome to this class. Today is class 7 of a series of 12 classes to learn advanced SLua.

[11:01] Suzanna: Today we are going to write a script to use a flat vector, a vector in a plane or 2D vector, a vector with two coordinates: x and y.

[11:02] Suzanna: Please click on the doggie Dufa at my side to receive the a folder with the Class Materials. At the end of the class Dufa will give you another folder with the class text and the suggested practices.

[11:02] Suzanna: I welcome questions and comments! Please stop me to ask a question related to the contents of the class at any time. Please wait at questions time to ask more general questions related to the class.

[11:02] Suzanna: Ali and I are at the Scripting Study Group Area on Thursdays, Fridays and Saturdays to help you with your practices and scripts, comment about them, and answer your questions. You have a LM to the Study Group in your Class Materials. On Fridays, the students of this series of classes have priority, in case that there are many questions.

[11:03] Suzanna: In a previous class we studied a script for an extended table where we added functions useful for tables and also a metatable with the metamethod __index to tell where to find the functions.

[11:03] Suzanna: We used three tables: the table with the data, the table with our functions and the table with the metamethods.

[11:04] Suzanna: Today we will use these ideas to script a flat vector, but with some improvements.

[11:04] Suzanna: Instead of a table to store any data, we will use a structured data table to store a flat vector, with its two coordinates x and y.

[11:04] Suzanna: We will use only one table with the functions and one table with the metamethods. In the previous class, with the extended table script, we were creating new tables each time that we made an extended table and they take script memory.

[11:05] Suzanna: Each flat vector will be a different table with its data, the two coordinates of the vector.

[11:06] Suzanna: "Script flat vector 1".

```
1  local flatVector = {}
2
3  local flatVectorMt = {
4      __index = flatVector
5  }
6
7  function flatVector.new(x, y)
8      return setmetatable({
9          x = x,
10         y = y
11     }, flatVectorMt)
12 end
13
14 local myVec1 = flatVector.new(1, 2)
15 local myVec2 = flatVector.new(3, 4)
16
17 print(myVec1.x)  -- 1
18 print(myVec2.y)  -- 4
```

[11:06] Suzanna: Viewing script FV1 lines 1 to 18

[11:07] Suzanna: In line 1 we initialize our table to store the functions.

[11:07] Suzanna: We add to the table the function "new", lines 7-12, that expects two parameters with the coordinates.

[11:07] Suzanna: This function generates the table with the two coordinates representing a vector that we will assign to a variable.

[11:07] Suzanna: In lines 8-11, with:
    { x = x, y = y }
we are creating a new table.

[11:08] Suzanna: From a LSL point of view this syntax looks strange, like assigning something to itself.

[11:08] Suzanna: But the left part of the = is the table key, not a variable. Another way to write it is:
    { ["x"] = x, ["y"] = y }

[11:08] Suzanna: We use the function setmetatable() to set the metatable of this new table to flatVectorMt.

[11:09] Suzanna: And we return the data table with the coordinates x, y and the metatable linked to it.

[11:09] Suzanna: In lines 3-5 we create the metatable, with the metamethod __index linking to flatVector, our functions table.

[11:09] Ali: In Lua parlance x = x is syntactic sugar for the more complete form ["x"] = x

[11:09] Suzanna: In lines 14-15 we create two flat vectors, using the function "new" with flatVector.new() and the coordinates.

[11:10] Suzanna: Now we have two data tables, with the coordinates x and y, assigned to myVec1 and myVec2.

[11:10] Suzanna: Only one metatable, linked to both vectors by setmetatable().

[11:10] Suzanna: And only one functions table, linked to both vectors by the metamethod __index.

[11:11] Suzanna: Now we have a very boring flat vector, we can only set its coordinates and print them. Next we are going to add more content.

[11:11] Marie: should a vector not have both an origin and an end point? Or is the origin for all vectors in this code snippet considered to be 0,0 ?

[11:11] Sandy Burgess: Why do you call it "flatVector" ... what is flat? 2D?

[11:11] Suzanna: i leave Marie's question to Ali :)

[11:12] Suzanna: Let's add some functions to our functions table, and also a way to print it in full, in the format < 5, 3 >

[11:12] Ali: In general, Marie, vectors are expressed as an offset from the origin of the coordinate system.

[11:12] Suzanna: its a 2d vector, on the place

[11:12] Sandy Burgess nods. Thank you

[11:12] Suzanna: we could have done a 3d vector, but we already have it in the language

[11:12] Marie: not always, Ali, but let's assume they are in this case

[11:13] Marie: in case you want to add vectors, the origin of the wecond cound be the end of the first

[11:13] joel dawn: tensor?

[11:14] joel dawn: sry i will listen rather, learning)

[11:14] Suzanna: i will leave to Ali that write an example with a tensor

[11:14] joel dawn: nice

[11:14] joel dawn: for physic

[11:14] joel dawn:)

[11:14] Sandy Burgess: Yes, but each vector in itself and the resulting vector will always be the offset from the origin, Marie

[11:15] Marie: Continue, Suzanna, I'll have a philosophical discussion with Ali during the practice session :)

[11:15] Suzanna: aside of the name and possible use of this script, this is the basic structure to define an object in SLua, in the OOP sense of it

[11:15] Tyra: every vector space contains a zero vector, by axiom *ducks*

[11:16] Ali: They are expressed that way, To to describe a vector from point A to point B, you use two vectors. For instance the vector from the origin to B minus the vector from the origin to A equals the vector from A to B, Vb-Va = Vab. Or alternately, Va + Vab = Vb.

[11:16] joel dawn: y

[11:17] Suzanna: more theory onf vectors and tensors tomorrow at the study group, with Ali, and Tyra if she volunteers :)

[11:17] Suzanna: question for you all, where is the metatable used in this script?

[11:17] Marie: I guess I had a different math teacher :)

[11:17] joel dawn: as returning value?

[11:17] Liam Hoffen: when searching the x and y members of the vectors in the print statements? goes via __index for a response

[11:18] Suzanna: no, the x and the y are found in the tables myVec1 and myVec2, the instances of the object in the OOP sense

[11:19] Suzanna: so since they are already found, SLua doesn't trigger the __index metamethod

[11:19] Ali: Remember flatVector contains the functions, not the the data.

[11:19] Suzanna: in this script we are not really using the metatable, only setting it

[11:20] Suzanna: to have the structure ready for the next script when we will add more things

[11:20] Ali: So far, the only flatVector function we have is new, which we invoke directly, not through the metatable.

[11:20] Suzanna: the table flatvector will store the functions, but now only have the function :new() and we are calling new on flatvector to create a flatvector

[11:22] Suzanna: question, which one has the metatable, myVec1 or myvec2 ?

[11:22] joel dawn: 2

[11:22] Cass Vaher: both

[11:22] joel dawn: is rewrote?

[11:22] Basil: but the new method sets the metatable and link it to flatVector

[11:22] Marie: both

[11:23] Basil: to flatVectorMt

[11:23] Suzanna: both have the metatable, we are assigning the metatable in :new() and we are calling :new() once for each vector

[11:23] Suzanna: the :new() methods sets the metatable

[11:23] Suzanna: we have the metatable already linked to flatvector in line 4

[11:24] Ali: Not quite, Basil. new sets flatVectorMt for the new table just created by setting x and y, and thereby links the new table to flatVector.

[11:24] Suzanna: question, how many copies of the metatable do we have at the end of the script? 1, 2, or more?

[11:25] joel dawn: 1

[11:25] Marie: 1

[11:25] Suzanna: one, this is the interesting part of it, we have only one copy and we use it for all the vectors

[11:25] Suzanna: instead if making a copy for each one

[11:26] Suzanna: question, do we also have only one copy of the table flatVector?

[11:26] joel dawn: no

[11:26] joel dawn: for each new?

[11:26] Cass Vaher: no

[11:26] Liam Hoffen: yes

[11:26] Marie: no, one for each instance

[11:26] joel dawn: like as sub metatables..into main metatable?

[11:26] joel dawn: hm?

[11:26] joel dawn:)

[11:27] Suzanna: yes, we have only one copy of the table flatVector

[11:27] Suzanna: the one created in line 1, that we have the function assigned in lines 7-12

[11:27] Basil: but 2 instances of it

[11:27] Marie: ah, ok, misunderstood the question

[11:27] Suzanna: we are not duplicated the table flatvector in :new()

[11:28] Ali: Two references to the same table, Basil.

[11:28] Suzanna: in :new() we are creating a new table with x and y, returning this new table and assigning it to the vector variables

[11:29] Suzanna: in total, how many tables do we have at the end of the script?

[11:29] Liam Hoffen: 3

[11:29] joel dawn: 4?

[11:29] Marie: 3

[11:30] Suzanna: waiting for more answers, lets see which option wins ...

[11:30] Marie: flatVector, myVec1 and myVec2

[11:30] Cass Vaher: 4

[11:31] Avril Vallejo: 2

[11:31] Suzanna: 4: flatVector, flatVectorMt, myVec1, myVec2

[11:31] Basil: if the data in myVec1 and myVec2 don't hold the same values, they can't have the same reference.

[11:32] Marie: haha, that metatable!

[11:32] Suzanna: myVec1 has keys x and y with values 1 and 2

[11:32] Suzanna: myVec2 has keys x and y with values 3 and 4

[11:32] Suzanna: flatVector has the function new()

[11:32] Suzanna: flatVectorMt has the metamethod __index

[11:32] Ali: This is a case where spreading the function over multiple lines may be interfering with clarity. For me, this is more readable:
```
function flatVector.new(x, y)
    return setmetatable( {x=x, y=y}, flatVectorMt)
end--new
```

[11:33] joel dawn: thats advanced truly ... :

[11:33] Suzanna: a metatable is also a table, like any other, with keys and values

[11:34] Suzanna: its the use that we make of this table what makes it a metatable

[11:34] Suzanna: yes, this is advanced series :)

[11:34] joel dawn: yea)

[11:34] Suzanna: your questions about this script....

[11:34] joel dawn: i meant that other syntax ... i am learning still)

[11:35] Ali: The only difference in this case, Joel is the choice of whitespace. More a matter of style, than syntax per se.

[11:35] joel dawn: i see now..sry i skipped some ..apologies

[11:35] joel dawn: i see it now..looked shrinked version for me..

[11:36] Suzanna: Ali prefers it compact, i prefer it spacy, you choose your favorite one :)

[11:36] Suzanna: going to the next script ...

[11:36] joel dawn: yup its i meant maybe ..hard looked sryy.)

[11:36] Marie: I'm also for spacy

[11:36] Suzanna: "Script flat vector 2".

```lua
local flatVector = {}

local flatVectorMt = {
    __index = flatVector,

    __tostring = function(self)
        return "<" .. self.x .. ", " .. self.y .. ">"
    end
}

function flatVector.new(x, y)
    return setmetatable({
        x = x,
        y = y
    }, flatVectorMt)
end

function flatVector.magnitude(self)
    return math.sqrt(self.x * self.x + self.y * self.y)
end

function flatVector.normalize(self)
    local mag = self.magnitude(self)
    if mag == 0 then
        return flatVector.new(0, 0)
    end
    return flatVector.new(self.x / mag, self.y / mag)
end

function flatVector.distanceTo(self, other)
    local dx = self.x - other.x
    local dy = self.y - other.y
    return math.sqrt(dx * dx + dy * dy)
end

local myVec1 = flatVector.new(1, 2)
local myVec2 = flatVector.new(3, 4)

print(myVec1)  -- <1, 2>
print(myVec2)  -- <3, 4>
print(myVec1.x)  -- 1
print(myVec2.y)  -- 4
print(myVec1.magnitude(myVec1))  -- 2.2360679774998
```

```
44 print(myVec2.normalize(myVec2))  -- <0.6, 0.8>
45 print(myVec1.distanceTo(myVec1,myVec2))  -- 2.8284271247462
```

[11:37] Suzanna: Viewing script FV2 lines 1 to 45

[11:37] Suzanna: Let's add some functions to our functions table, and also a way to print it in full, in the format < 5, 3 >

[11:37] Ali nods. "Use the style you find most readable, or that conforms to the standards of a group if you're working with one. I definitely come out on the dense end of the spectrum." She laughs, "In fact, coding for myself, I'd put the whole business on one line instead of three."

[11:37] Suzanna: The first function is magnitude() in lines 18-20.

[11:38] joel dawn: got it)

[11:38] Suzanna: We have only one flatVector table with the functions. All the vectors will be calling this table, so we need to know what vector is calling the function.

[11:38] Suzanna: We receive it in the parameter "self". It will contain the data table (with the coordinates) of the vector that is calling magnitude().

[11:39] Suzanna: We are calling it "self" because is the SLua convention.

[11:39] Suzanna: magnitude() uses the function sqrt() in the math library to return the value.

[11:39] Suzanna: Next function is normalize(), in lines 22-28, also using the parameter "self".

[11:40] Suzanna: In this function we return a flat vector. The functions calls "new" to create a new one in line 25 or 27.

[11:40] Ali: the use of self in the way is inherited from object oriented programming more generally

[11:40] Suzanna: And distanceTo(), in lines 30-34.

[11:40] Suzanna: These function needs two flat vectors. We have the data table of the one calling the function in "self". And the data table of the other one in "other". We could use any other name instead of "other".

[11:41] Suzanna: To print the vector formated as < 5, 3 > , there is another metamethod: __tostring().

[11:41] Suzanna: __tostring() is called when we change a variable to a type string, for instance with tostring(). The function print() also calls __tostring().

[11:42] Suzanna: We need to add a __tostring metamethod to say how our table is converted to string. This metamethod must return a string.

[11:42] Suzanna: We only have one metatable for all the vectors, as with the functions, we need the data table of the vector, and we receive it in "self".

[11:43] Suzanna: Looking at the tests, we need to use:
myVec1.magnitude(myVec1)
repeating myVec1 to send it to the function in the parameter "self".

[11:43] Suzanna: This is not nice, we will solve it in next script.

[11:44] Suzanna: questions about this script ...

[11:44] Suzanna: question for you, what datatype is that parameter "self"?

[11:45] joel dawn: number

[11:45] Liam Hoffen: flatVector

[11:45] Marie: vector

[11:45] Marie: flatvector

[11:45] Suzanna: all wrong, try again ...

[11:45] Ali: not quite.

[11:45] Avril Vallejo: like a pointer?

[11:45] Ali: What is self doing?

[11:46] Sandy Burgess: table

[11:46] Ali: Right direction Aviril

[11:46] Suzanna: i mean, what will be typeof(self) ?

[11:46] Liam Hoffen: ahh....table

[11:46] joel dawn: nil?

[11:46] joel dawn: oh ok

[11:46] Suzanna: a table, self is a table

[11:46] Suzanna: which table?

[11:47] Liam Hoffen: a table modeled after flatVector

[11:47] Suzanna: and in this example?

[11:47] joel dawn: metatable ?

[11:47] Marie: {1, 2} or {3, 4}

[11:48] Sandy Burgess: Seems we are all left to guessing. I miss last session and have the feeling that it is the missing piece in this puzzle

[11:48] Suzanna: myVec1 or myVec2, the tables that we have created when calling :new(), or their values as Marie says

[11:48] Basil: the 2 vectors are of type table

[11:48] Suzanna: yes, Basil

[11:48] Marie: last session was not about metatables

[11:49] Suzanna: we studied metatables in class 2 and class 3

[11:49] Ali: self is a reference, to a table containing our vector data in this case, hence of type "table"

[11:49] Marie: diggs deep in her memory baanks ...

[11:49] Sandy Burgess follows Marie and finds a void

[11:49] Marie: lol

[11:50] Suzanna: when we call a function, like magnitude(), we need to pass the vector from which we want the magnitude

[11:50] Ali: I don't think we discusses the use of self references in those lessons. This is a new concept for you, I think?

[11:50] Suzanna: yes, we are studying self for the first time today

[11:51] Suzanna: because there is only one function magnitude() that must work with any of the vectors

[11:51] Suzanna: so it needs to know which vector

[11:52] Ali: This gets into the magical difference between the dot syntax and the colon syntax when calling table functions.

[11:52] Suzanna: that we will see next

[11:52] Suzanna: questions about this script ...

[11:52] Suzanna: going on ...

[11:52] Suzanna: "Script flat vector 3".

```lua
local flatVector = {}

local flatVectorMt = {
    __index = flatVector,

    __tostring = function(self)
        return "<" .. self.x .. ", " .. self.y .. ">"
    end
}

function flatVector:new(x, y)
    return setmetatable({
        x = x,
        y = y
    }, flatVectorMt)
end

function flatVector:magnitude()
    return math.sqrt(self.x * self.x + self.y * self.y)
end

function flatVector:normalize()
    local mag = self:magnitude()
    if mag == 0 then
        return flatVector:new(0, 0)
    end
    return flatVector:new(self.x / mag, self.y / mag)
end

function flatVector:distanceTo(other)
    local dx = self.x - other.x
    local dy = self.y - other.y
    return math.sqrt(dx * dx + dy * dy)
end

local myVec1 = flatVector:new(1, 2)
local myVec2 = flatVector:new(3, 4)

print(myVec1)  -- <1, 2>
print(myVec2)  -- <3, 4>
print(myVec1.x)  -- 1
print(myVec2.y)  -- 4
print(myVec1:magnitude())  -- 2.2360679774998
```

```
44 print(myVec2:normalize())  -- <0.6, 0.8>
45 print(myVec1:distanceTo(myVec2))  -- 2.8284271247462
```

[11:52] Suzanna: Viewing script FV3 lines 1 to 45

[11:53] Sandy Burgess: And out of the void there tumble some bits and pieces of the puzzle

[11:53] Suzanna: This script does exactly the same, but more glamorously.

[11:53] Suzanna: Now, in line 43, we use:
myVec1:magnitude()

[11:53] Suzanna: We don't send myVec1 as parameter, and we use ":" instead of "." to call the function.

[11:54] Ali: and more succinctly, and with fewer backreferences to resolve.

[11:54] Suzanna: The colon tells SLua to automatically add the myVec1 to the function call as the first parameter.

[11:54] Sandy Burgess: Why does myVec1.function not have access to the values of myVec1?

[11:55] joel dawn: it has no reference on itself?

[11:55] Suzanna: with "." we call the function on myVec1, but the function can't know where it has not been called from

[11:56] Suzanna: with the "." syntax we need to pass myVec1 as parameter

[11:56] Sandy Burgess: Because its in the metatable and the metatable does not know the table that is using it atm?

[11:56] Suzanna: a function has no reference on what has called it

[11:56] Ali: When you execute the function, that way, no information about the which table it was called from is passed. This is the whole purpose of the self reference, to provide that information.

[11:57] Sandy Burgess: A few more puzzle pieces appear ...

[11:58] Marie: In the . notation, it's esy to see where self comes from; not so much in the : notation where one needs to have the background about how lua deals with that : notation.

[11:58] Suzanna: with the ":" syntax, we tell SLua that we want the function to be called with myVec1 or with myVec2, in general with the table that is at the left of ":", and SLua arranges it for us

[11:58] Sandy Burgess: And now self automatically knows that it is refferencing the calling vector/table?

[11:58] joel dawn: it takes whole call as one at once..?

[11:58] Suzanna: with ":" we are doing the same than with "," but in an abreviated way

[11:59] joel dawn: due references

[11:59] Marie: Coders without that knowledge, stumbling on a Lua script without previous Lua knowledge, will not understand how welf gets a value

[11:59] Suzanna: SLua internally creates a self variable in the called function, with the table on which the function has been called from

[11:59] Marie: self*

[12:00] joel dawn: so its gate for return ..

[12:01] Suzanna: When calling a function in a table with ":", the table is sent as the first parameter to the function.

[12:01] Suzanna: Off course if we use two vectors in the function, we still need to send the second one:
```
myVec1:distanceTo(myVec2)
```

[12:01] Suzanna: The colon not only works when calling a function, it also works when defining a function.

[12:02] Suzanna: Let's look at magnitude() in lines 18-20.

[12:02] Suzanna: Now we have the ":", we don't have the parameter "self", but anyway we are using self in line 19.

[12:02] joel dawn: can we use that for position of myvec1, when if myVec2 is void..like as from start point?

[12:02] Sandy Burgess: Could we use it in __tostring = self:function ...

[12:02] joel dawn: like as distance from 0,0

[12:03] Ali: When we use : in a function definition, the variable self is automatically prepended to our argument list. When we use : in a function call, the first argument to the function (normally self) is set a reference to whatever is before the :

[12:03] Suzanna: yes, Sandy, we will rewrite __tostring in a next script to use :

[12:04] Suzanna: When we use the colon in a function definition, the first parameter is automatically placed in the variable "self".

[12:04] Sandy Burgess smiles about more puzzle pieces: Brain is waking up slowly

[12:04] Suzanna: Using ":" makes the code cleaner and saves on typing.

[12:05] Suzanna: open script2 and script3 together to compare

[12:05] Marie: but more difficult to read and interpret

[12:05] joel dawn: maybe it consumes more memory

[12:05] Suzanna: in script2 we have: `print(myVec1.magnitude(myVec1))`

[12:05] Sandy Burgess: Not once you are used to it, Marie - its commonly used in many languages

[12:05] Suzanna: in 3 : `print(myVec1:magnitude())`

[12:05] Marie: it asks more effort from my neurons

[12:06] Suzanna: in 2 we have: `function flatVector.magnitude(self)`

[12:06] Suzanna: in 3 : `function flatVector:magnitude()`

[12:07] Marie: and self magically gets a value!

[12:07] Sandy Burgess: Can you please remind me what the metatable __index does - that threw me off from the beginning.

[12:07] Suzanna: in script 2 we are passing myVec1 as parameter, and receiving it in the parameter self

[12:07] joel dawn: its trigger to pass value to function ... looks for me

[12:07] Ali: It also has the important effect of evaluating whatever expression precedes the : once instead of twice. Normally, this will save a bit of time and memory, but will otherwise be operationally the same, however in some circumstances, when the expression before the : has side effects, then there will be an operational difference in that the side effect will occur once with the : syntax and twice with the . syntax.

[12:07] Suzanna: in script 3 we are not passing a parameter, and not receiving it, but "self" appears

[12:08] Marie: magic! :)

[12:08] Suzanna: because of the : notation, first we tell SLua to pass the table at the left of the : as a first parameter, and in the function we tell SLua to take that first parameter and copy it to self

[12:09] Basil: could we do function flatVector.normalize(self)?

[12:09] Sandy Burgess: Ah - so script 2 would count up twice with a counter

[12:09] Ali: Yes, Basil and the outcome would be exactly the sam.

[12:09] Suzanna: the metamethod __index is called when a value doesn't exist in the table

[12:10] Suzanna: if we use myVec1.x myVec1 has the key "x" and the metatable is not used

[12:10] Suzanna: when we use myVec1.magnitude the function magnitude is not in myVec1 Slua calls the __index metamethod in its metatable

[12:10] Sandy Burgess: Why do we add __index if it does not contain any data?

[12:11] Suzanna: and the table, flatVector, in __index tells SLua to look there for magnitude

[12:11] joel dawn: default values?

[12:11] joel dawn: no error or..

[12:11] Suzanna: __index tells SLua where to look for the data, in this example in the table flatvector that has the functions

[12:11] Ali: using the . syntax instead of the : syntax during a function declaration makes no difference, except that it allows you to name the first variable something besides self if you really wanted to.

[12:12] Sandy Burgess: And a whole chandelier lightens up

[12:12] Suzanna: yes, Basil, flatVector.normalize(self) also works

[12:12] Marie: In script 3, can we give 'self' a different name? Without Slua getting confused?

[12:13] Ali cheers for Sandy.

[12:13] Suzanna: using the : notation the parameter is predefined to be "self", SLua names it "self", we can't change it

[12:13] Ali: That's right Marie.

[12:13] Marie: ok, it would be real magic if it could be changed

[12:13] Ali: Ot

[12:14] Sandy Burgess: then use flatVector.magnitude(myVariableName)

[12:15] Sandy Burgess: with that call you can name your variable whatever you want

[12:15] Suzanna: yes, we can do it, but it's more usual to use the : notation and "self"

[12:15] Basil: Why would we change it Marie? how better name it?

[12:15] Ali: It is fine to declare the function with the . syntax and invoke with the : syntax. I wouldn't recommend it from the point of view of readability, but your code will be compile to exactly the same thing.

[12:16] Sandy Burgess: ... or just get used to new syntax options

[12:16] Ali: Basil, on occasion, when dealing with binary operators, I've seen left, used instead of self and right as the other variable.

[12:16] Marie: changing it would set SLua on the wrong foot; it's the internal SLua magic. But I wanted to make this clear

[12:17] Basil: oh

[12:17] Basil: left meaning self?

[12:17] Suzanna: new we will see binary operators, where we not use self

[12:17] Suzanna: next*

[12:17] joel dawn: its as keyword then

[12:17] Ali: Yes, having the same use as self in that context.

[12:17] Free Radar HUD v1.1 by Crystal Gadgets

[12:17] Suzanna: But we also want to do operations with flat vectors, like adding two of them.

[12:18] Suzanna: We have metamethods to tell SLua what to do when two flat vectors are added, like myVec1 + myVec2 .

[12:18] Ali: no, joel, more like a convention that SLua presumes.

[12:18] joel dawn: ok

[12:18] Suzanna: SLua calls the metamethod __add when it finds and operator +.

[12:18] joel dawn: makes it more lively)

[12:18] Suzanna: We have to assign a function to __add that returns the sum of two flat vectors.

[12:18] Suzanna: "Script flat vector 4".

```lua
local flatVector = {}
local flatVectorMt = {}

function flatVector:new(x, y)
    return setmetatable({
        x = x,
        y = y
    }, flatVectorMt)
end

function flatVector:magnitude()
    return math.sqrt(self.x * self.x + self.y * self.y)
end

function flatVector:normalize()
    local mag = self:magnitude()
    if mag == 0 then
        return flatVector:new(0, 0)
    end
    return flatVector:new(self.x / mag, self.y / mag)
end

function flatVector:distanceTo(other)
    local dx = self.x - other.x
    local dy = self.y - other.y
    return math.sqrt(dx * dx + dy * dy)
end

flatVectorMt = {
    __index = flatVector,

    __add = function(left, right)
        return flatVector:new(left.x + right.x, left.y + right.y)
    end,

    __sub = function(left, right)
        return flatVector:new(left.x - right.x, left.y - right.y)
    end,

    __mul = function(left, right)
        if type(right) == "number" then
            return flatVector:new(left.x * right, left.y * right)
        elseif type(left) == "number" then
```

```
44                 return flatVector:new(right.x * left, right.y * left)
45             else
46                 return left.x * right.x + left.y * right.y
47             end
48         end,
49
50         __tostring = function(self)
51             return "<" .. self.x .. ", " .. self.y .. ">"
52         end
53     }
54
55     local myVec1 = flatVector:new(1, 2)
56     local myVec2 = flatVector:new(3, 4)
57
58     print(myVec1)  -- <1, 2>
59     print(myVec2)  -- <3, 4>
60     print(myVec1.x)  -- 1
61     print(myVec2.y)  -- 4
62     print(myVec1:magnitude())  -- 2.2360679774998
63     print(myVec2:normalize())  -- <0.6, 0.8>
64     print(myVec1:distanceTo(myVec2))  -- 2.8284271247462
65     print(myVec1 + myVec2)  -- <4, 6>
66     print(myVec1 - myVec2)  -- <-2, -2>
67     print(myVec1 * 3)  -- <3, 6>
68     print(myVec1 * myVec2)  -- 11
```

[12:19] Suzanna: Viewing script FV4 lines 1 to 68

[12:19] Marie: ok, Ali, if we can use a different word in script 3, how will Slua know which name we use instead of self?

[12:19] Suzanna: with : notation, in script 3, we can only use self

[12:19] Suzanna: with . notation, in script 2, we can choose the name

[12:20] Marie: yes, but Ali said the contrary, hence my confusion

[12:20] Suzanna: i suggest to use : notation and self

[12:20] Ali: Because it will be the first argument in our argument list, here, Like with our _add function now.l

[12:20] Suzanna: it would be a mix of script 2 and script 3

[12:21] Marie: let's continue this in the practice sessions, Ali

[12:21] Suzanna: We have it in lines 32-34.

[12:21] Suzanna: We receive two parameters, the vectors at each side of the +.

[12:21] Suzanna: In line 33 we create a new vector with the coordinates added and return it as the result of addition.

[12:22] Suzanna: The metamethod for subtraction is __sub. We have it in lines 36-38.

[12:22] Ali grins. I" haven't disagreed with Suzanna yet on this. At least not on purpose. Yes, we can clarify later."

[12:22] Suzanna: And __mul, to multiply, in lines 40-48.

[12:22] Suzanna: This one is longer because we are doing to different operations:
    flatVector * number: to scale the flatVector, returning a new flatVector.
    flatVector * flatVector: to calculate the dot product, returning a number.

[12:23] Suzanna: There is no way to define two different __mul metamethods with different parameters.

[12:23] Suzanna: SLua parameters has no type, they are always "left" and "right". Or any other names that you prefer, often "a" and "b" is used.

[12:23] Suzanna: If we write a second __mul = function(left, right) , it will just replace the previous one in the table.

[12:24] Suzanna: We need to do it in one function, looking at the type of the parameters, with type().

[12:24] Suzanna: If "right" is a number, we return a new vector scaled, lines 41-42.

[12:24] Suzanna: It could also be number * flatVector so we are checking if "left" is a number to do the same, lines 43-44.

[12:25] Suzanna: Otherwise we return the dot product in line 46.

[12:25] Suzanna: We are not checking that the other operand is a flat vector, or that both operands are. Neither in __add or __sub.

[12:25] Anton: I don't understand. What is the datatype of left or right when it's not a number?

[12:25] Suzanna: SLua philosophy is oriented to speed and saving memory. Adding checks is against it. SLua assumes that the scripters are clever and call functions with the right parameters.

[12:26] Marie: could also be a table, Anton

[12:26] Ali: Anton our vector table

[12:26] Anton: i see. thanks

[12:26] Ali: Or else we get a runtime error

[12:26] Suzanna: we can multiply a flatvector by a number, or two flatvectors, the types would table * number or number * table or table * table

[12:27] Suzanna: The idea is that is more efficient not to add many checks and leave to the scripter to use the functions well, tather than using more scripting time in checks.

[12:27] Anton: I'm not sure I understand multiplying a number by a table.

[12:27] Ali: Also, because of the way _add is invoked, one or the other of the arguments _must_ be a table with flatVectorMt as it's metatable.

[12:28] Basil: 3 * {2, 5} = {6, 15}

[12:28] Suzanna: usually we can't multiply numbers with tables, so in the __mul metamethod we add the code to do the multiplication, multiplying each of the two components of the flatvector by the number

[12:29] Anton: Thanks, Basil. Now I understand

[12:29] Ali: It's multiply in a vector by a scalar, Anton. new.x = oldx*s, new.y = old.y*s

[12:29] Anton: thanks

[12:29] Suzanna: If we use a wrong parameter, probably we will get a runtime error because it will not have keys with "x" and "y". Unless we use a SLua vector or rotation, that it will work, in some way.

[12:29] Ali: or any other table with x and y keys that return numbers.

[12:30] Suzanna: If we want to check that it is a flat vector, we can use getmetatable():
```
    if getmetatable(left) == flatVector then  -- it's a flat vector
```

[12:30] Suzanna: The metamethods for arithmetic operations are:
```
    __add (+), __sub (-), __ mul (*), __div (/), __idiv (//), __mod (%),
__pow (^)
```

[12:31] Suzanna: For strings:
```
    __concat (..)
```

[12:31] Suzanna: And array tables:
```
    __len (#)
```

[12:31] Suzanna: There are also comparison operators:
```
    __eq (==), __lt (<), __le (≤)
```
the other operators are calculated from these ones (~=, >, ≥).

[12:31] Ali: Note that nothing, except good sense, prevents us from adding other data to our vectors. If we had some need, we could add other keys to the table, and these operations would cheerfully ignore them.

[12:32] Suzanna: questions about defining operators ...

[12:33] Suzanna: going on ...

[12:33] Suzanna: Let's do another improvement. We are using one table for our functions and another table for the metamethods.

[12:33] Suzanna: But the metamethods all start with __, and our functions, by convention, should not.

[12:33] Suzanna: So we can place everything in only one table.

[12:34] Suzanna: "Script flat vector 5".

```lua
local flatVector = {}

function flatVector:new(x, y)
    return setmetatable({
        x = x,
        y = y
    }, flatVector)
end

function flatVector:magnitude()
    return math.sqrt(self.x * self.x + self.y * self.y)
end

function flatVector:normalize()
    local mag = self:magnitude()
    if mag == 0 then
        return flatVector:new(0, 0)
    end
    return flatVector:new(self.x / mag, self.y / mag)
end

function flatVector:distanceTo(other)
    local dx = self.x - other.x
    local dy = self.y - other.y
    return math.sqrt(dx * dx + dy * dy)
end

flatVector.__index = flatVector

function flatVector.__add(left, right)
    return flatVector:new(left.x + right.x, left.y + right.y)
end

function flatVector.__sub(left, right)
    return flatVector:new(left.x - right.x, left.y - right.y)
end

function flatVector.__mul(left, right)
    if type(right) == "number" then
        return flatVector:new(left.x * right, left.y * right)
    elseif type(left) == "number" then
        return flatVector:new(right.x * left, right.y * left)
    else
```

```
44            return left.x * right.x + left.y * right.y
45        end
46    end
47
48    function flatVector:__tostring()
49        return "<" .. self.x .. ", " .. self.y .. ">"
50    end
51
52    local myVec1 = flatVector:new(1, 2)
53    local myVec2 = flatVector:new(3, 4)
54
55    print(myVec1)  -- <1, 2>
56    print(myVec2)  -- <3, 4>
57    print(myVec1.x)  -- 1
58    print(myVec2.y)  -- 4
59    print(myVec1:magnitude())  -- 2.2360679774998
60    print(myVec2:normalize())  -- <0.6, 0.8>
61    print(myVec1:distanceTo(myVec2))  -- 2.8284271247462
62    print(myVec1 + myVec2)  -- <4, 6>
63    print(myVec1 - myVec2)  -- <-2, -2>
64    print(myVec1 * 3)  -- <3, 6>
65    print(myVec1 * myVec2)  -- 11
```

[12:34] Suzanna: Viewing script FV5 lines 1 to 65

[12:35] Suzanna: We move the metamethods to the table flatVector and stop using flatVectorMt.

[12:35] Suzanna: In the function "new", lines 3-8. We set the metatable to flatVector.

[12:35] Suzanna: From lines 28 to 50 we are adding all the metamethods to flatVector.

[12:35] Suzanna: In __tostring(), lines 48-50, we are using the ":", so no "self" parameter.

[12:36] Suzanna: Operator metamethods don't have a "self". They have the operands. They can't use the ":" and stay with the ".".

[12:37] Suzanna: its the same than before, but instead of using two table (flatVector and flatVectorMt) we have the functions and metatables in the same table (flatVector)

[12:37] Marie: this is where using that : gets confusing

[12:38] Suzanna: this script is the usual way to create an object in SLua, in the OOP sense of it

[12:38] Suzanna: questions about this script ...

[12:38] Marie: so, we are using meetamethods without a metatable?

[12:39] Suzanna: no, we are using metamethods with a metatable, flatVector, that we assign in line 7

[12:39] Suzanna: and flatVector also have the functions like magnitude and normalize

[12:39] Ali: No marie, we are using a metatable and adding other things besides metamethods too it.

[12:40] Marie: scrolling up, and yes, there's the metatable! :)

[12:40] Suzanna: flatVector us storing both metamethods and functions

[12:40] Suzanna: is*

[12:41] Suzanna: there is another script 6, but i will skip it today, and we will see it another day, or you can read it in the notes if you are confident with scripts 1 to 5

[12:42] Ali: What we are doing away with is the __index that passes references to another table and instead, resolving those references in the current table.

[12:42] Marie: so the metatable is itself?

[12:42] Suzanna: the metatable is flatVector, and __index tells SLua to look for the function also in flatVector

[12:42] Ali: __index now points to the metatable instead of another table.

[12:43] Suzanna: flatVector is metatable and functions table at the same time

[12:43] Marie: ok, thx Suzanna

[12:44] Basil: To be sure, with the class table functions we can use colon and dot and for the metamethods we use dot?

[12:44] Ali: basically we've renamed flatVectorMt to flatVector, and merged the old flatVector into the new one.

[12:45] Suzanna: we can use colon with __string

[12:45] Suzanna: but most metamethods are dot only

[12:46] Suzanna: __tostring *

[12:46] Marie: this is what this mixed notation makes SLua syntax difficult

[12:47] Suzanna: it makes SLua interesting :)

[12:47] Ali: This particular notation is common in a much broader world than Lua.

[12:47] Marie: yes, but not everyone is a scripting polyglot, Ali :)

[12:47] Marie: it is confusing in any language

[12:48] Basil: interesting but needs more memory, I mean more programmer memory

[12:48] Marie: I fail to see the logic in it

[12:48] Sandy Burgess: Programmer memory is less limited than script memory

[12:48] Basil: haha

[12:48] Ali laughs. "No. I'm Just spreading the blame. The . vs : syntax isn't a Lua peculiarity, it's bigger than that."

[12:48] joel dawn: i see it we pick only one parameter or whole..in case ours twos

[12:48] Anaïs: you don't know my memory....

[12:49] Marie: lol

[12:49] Sandy Burgess: I am sure its larger than 128

[12:49] Marie: mine is FIFO (first in, First out)

[12:49] Basil: lol

[12:49] Anton has difficulty remembering what he had for breakfast, sometimes

[12:49] joel dawn: thats best))

[12:49] joel dawn: out first then in

[12:49] joel dawn:)

[12:49] joel dawn:)sry..

[12:50] joel dawn: went to meta..okok

[12:50] Suzanna: And we have finished with the new scripts for today.

[12:50] Suzanna: Please click on the doggie Dufa at my side to receive the Class Notes. You will receive a folder containing the text of the class.

[12:50] Suzanna: And a notecard "Builders Brewery Class Evaluation Form" to evaluate the teachers and the class, as usual in all BB classes. Please, fill it and drop it in the red post box on my left.

[12:50] Avril Vallejo: nice class ..thnaks all

[12:50] joel dawn: Thank you for class

[12:51] Avril Vallejo: hugs

[12:51] Sandy Burgess: Thank you Ali and Suzanna

[12:51] Anaïs: Not to brag but I can forget what I'm doing while I'm doing it

[12:51] joel dawn: i is just example what all we can do it

[12:51] Marie: lol, Anaïs! :)

[12:51] joel dawn: we maybe can shrink it on special use..

[12:51] xaka Chayoo: thank you for the lesson

[12:51] Cass Vaher: thanks Suzanna and Ali

[12:51] joel dawn: its cool example..it shows all possibility of that function..like as multiplication does

[12:52] joel dawn: for me..idk when if i see all)

[12:52] Suzanna: thanks for coming, remember to come to the study groups too :)

[12:52] joel dawn: and garbage collector deletes all news?

[12:52] Marie: I foresee major headaches developing the practices

[12:52] Anton: Thanks Suzanna and Ali. Bye for now

[12:52] joel dawn: not as in other OOP where new had destructor..

[12:52] xaka Chayoo: see you

[12:52] Cass Vaher: ○°☆"Bye Bye ^·^"☆°○

[12:53] joel dawn: thank you again see you

[12:53] Ali: Marie, the difference became important for reasons I explained earlier. With the : syntax you evaluate a potentially complex expression with potential side effect once. With the dot syntax, you evaluate it twice. Now a modern, smart compiler might possibly be able to compensate, except that might break things where you were depending on the double side effect such that the second expression evaluated to something different. But back in the day, compilers generally weren't expected to be that smart anyway, so the : syntax was invented to make it possible for a programmer to explicitly choose between one evaluation or two.

[12:53] Marie: SLua only mimics OOP behavior, using those dreaded metatables

[12:53] Basil: thanks for the class Suzanna and Ali. Again Intensive and very interesting

[12:54] Sandy Burgess: Imagine this:
counter.functionCall(counter)
This would count up twice

[12:54] Marie: mkes me wonder if SLua was made in the dark middle ages of coding

[12:54] joel dawn: it looks as assembler

[12:55] Marie: it's not particularly bright and it's full of contradictions and confusing terminology

[12:55] Anaïs: your chat logger thingy failed about a half hour in. Now I need remember to export from my own logs. <makes a note>

[12:55] Ali: Oh yikesl

[12:56] joel dawn: its a way to reduce code to needs only

[12:56] joel dawn: its nice ..)

[12:56] joel dawn: not as others API

[12:56] joel dawn: see you

[12:56] Anaïs: yeah I save those off to a pdf for memory refershes, since y'all go into more detail than the actual class notes.

[12:56] Sandy Burgess: I like it

[12:56] joel dawn: it will be scripting too

[12:57] joel dawn: for best

[12:57] joel dawn: i see it its Slua ready for

[12:57] joel dawn: idk all but i will not too..its modern and most effecient..in promises..we will see

[12:57] joel dawn: both works but only one is true good

[12:57] joel dawn: ok i go to recheck all .. :)

[12:58] Ali: Thank you all for your very kind tips. And thanks also for supporting the Builder's Brewery.

[12:59] Ali: Thanks too for your enthusiastic participation.

[12:59] Marie: Byee everyone, CU all tomorrow or one of the following days at the practice sessions

[12:59] Suzanna: see you :)

[12:59] Basil: Practices will be intense

[13:00] Ali: Grr

[13:00] Marie: should I take some headache medicine in advance,

[13:00] Marie: ?

[13:00] Basil: No, bring your questions and sens of humor

[13:00] Marie: lol Basil

[13:01] Ali laughs. "I hope not. Maybe I should make some point and vector objects for us to play with."

[13:01] Suzanna: the chat to pdf gadget has failed, i will send you a link to the pdf tomorrow, who does use it? Ali, Anaïs, Tyra ...

[13:01] Ali: Thanks, Suzanna.

[13:02] Marie: looking at the practices and remembering how I solved them (or not) the previous run of this series, I feel cold sweat coming up ...

[13:02] Anaïs: the chat capture itself seems to have failed entirely. the last entry I have on screen is at [11:24]

[13:02] Marie: Anyway, it's late evening now and I will sleep over it and look at the practices tomorrow

[13:03] Marie: Byee everyone :)

[13:03] Tyra: I had the same issue with last week's, Anaïs

[13:03] Anaïs: but yes, if you had a something still getting it all, I'd appreciate the link when you can °◡° thank you

[13:03] Basil: bye Marie

[13:03] Tyra: byee Marie

[13:03] Ali: "Sleep well. Dream vector dreams.

[13:03] Basil: hummm

[13:03] Suzanna: see you :)

[13:03] Marie: lol, Ali, counting vectors instead of sheep :)

[13:03] Anaïs: but it is food time.

[13:04] Anaïs: have a good rest of the day all.

[13:04] Ali: Bye Anais.

[13:04] Basil: bye Anais

[13:05] Basil: Me too going cooking ...

[13:05] Basil: bye

[13:08] Ali: Bye basil

[13:08] Ali: Cooking with Basil. I do that a lot, actually.